

N-Queens

can make row-wise
column-wise, diagonal
-wise

same

→ row

→ column

→ diagonal

Q_1, Q_2, Q_3, Q_4

	1	2	3	4
1				
2				
3				
4				

x

column no.

Arrange 4 Queens, such that they are not under attack. [Find all the possible arrangement]

* Identification
means Back Tracking

[If like it would
have mentioned
optional then, it
would have for
to DP]

If $C_4 \rightarrow$ ways

↓
to reduce the size

Place 1st queen in first row
2nd queen in second row
3rd queen in third row

row under attack is solved. Now, what we have to decide is?
→ Column.

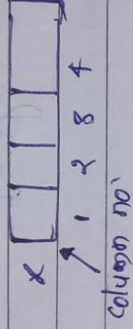
↓
[Now while keeping we will about
keeping in direct/single column]

these two are solved directly

Now, we will check for diagonal

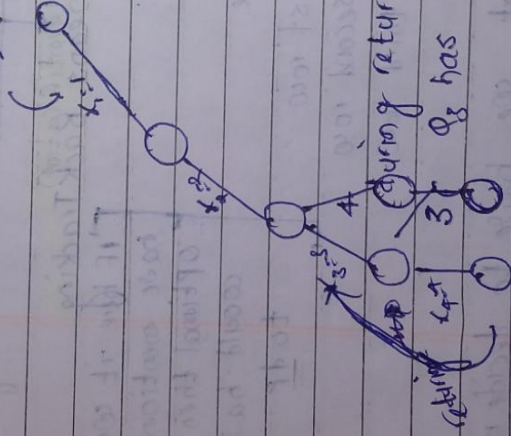
$$Q_1, Q_2, Q_3, Q_4$$

Q_1	Q_2	Q_3	Q_4
-------	-------	-------	-------



Column no.

State. space. Tree [Prepared by
column considering with
(attack



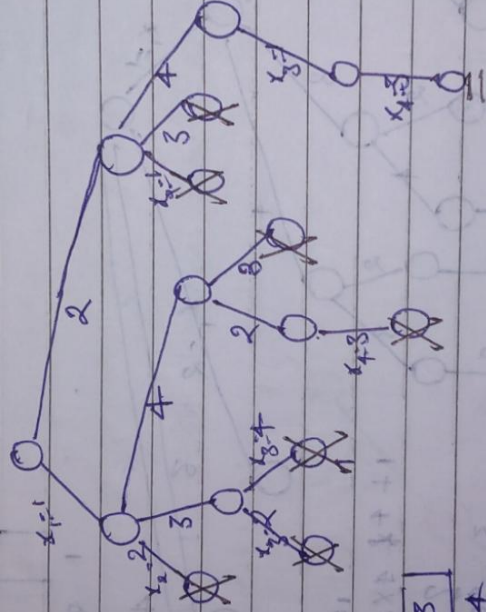
A 4x4 grid with handwritten labels Q_1 , Q_2 , Q_3 , and Q_4 in the bottom row.

Q.

1	3	3	4
1	Q ₁		
2		Q ₂	
3			
4			

state space Tree

by using Bounding $f \rightarrow$ row column diagonal



x	2	4	1	3
	1	2	3	4

3 1 4 2 $\rightarrow$ Next possible

Solⁿ:-

2, 4, 1, 3

3, 1, 4, 2 $\rightarrow$ Find by self

by Tracking

Q.

by Bounding

$$\sum_{i=1}^n w_i x_i$$

2	4
---	---

4000 OF Subsets

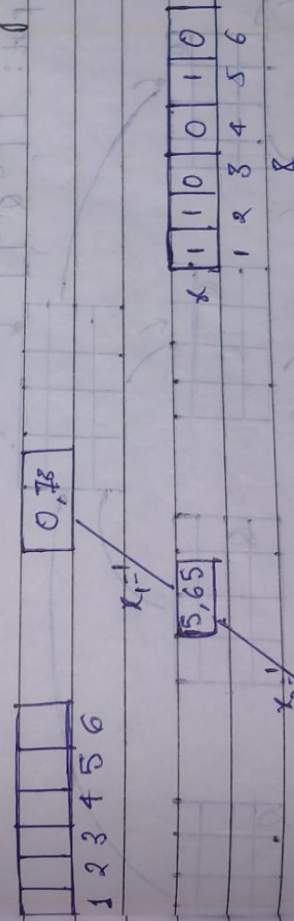
$$w[1:6] = \{5, 10, 12, 13, 15, 18\}$$

$$weights \quad n=6, \quad w=30$$

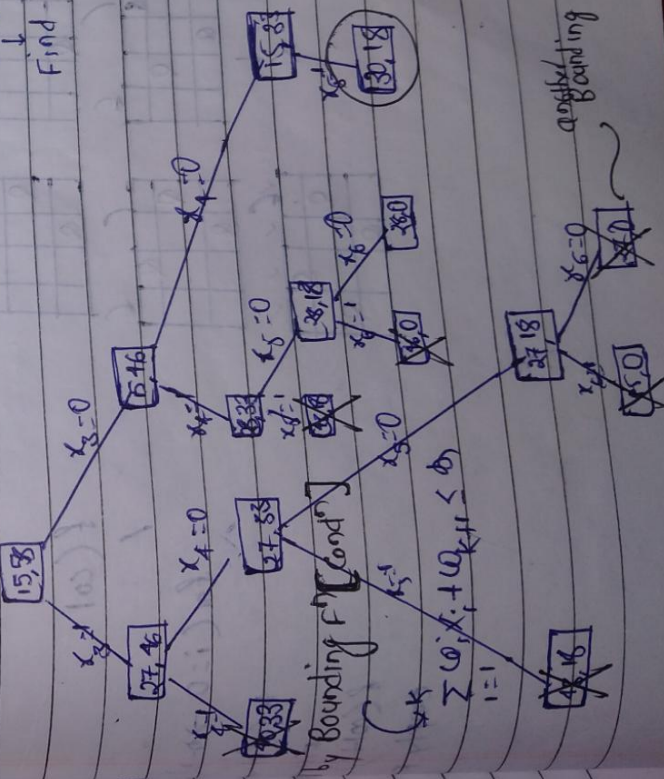
$$x_1 = 0/1, \quad 2 \quad 3 \quad 4 \quad 5 \quad 6$$

we have to take all the subset of those weight such that sum is 30

identification
Backtracking



Find by drawing Tree



$$Bouding \quad \sum_{i=1}^k w_i x_i + \sum_{i=k+1}^n w_i$$

so far included

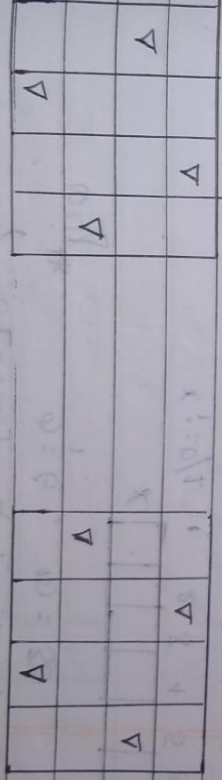
remaining

Graph Colouring

Hamiltonian cycle

Branch & Bound, NP-Hard, string algorithm

Q. Place 8 queens on 8x8 chessboard, so that no two queens attack each other



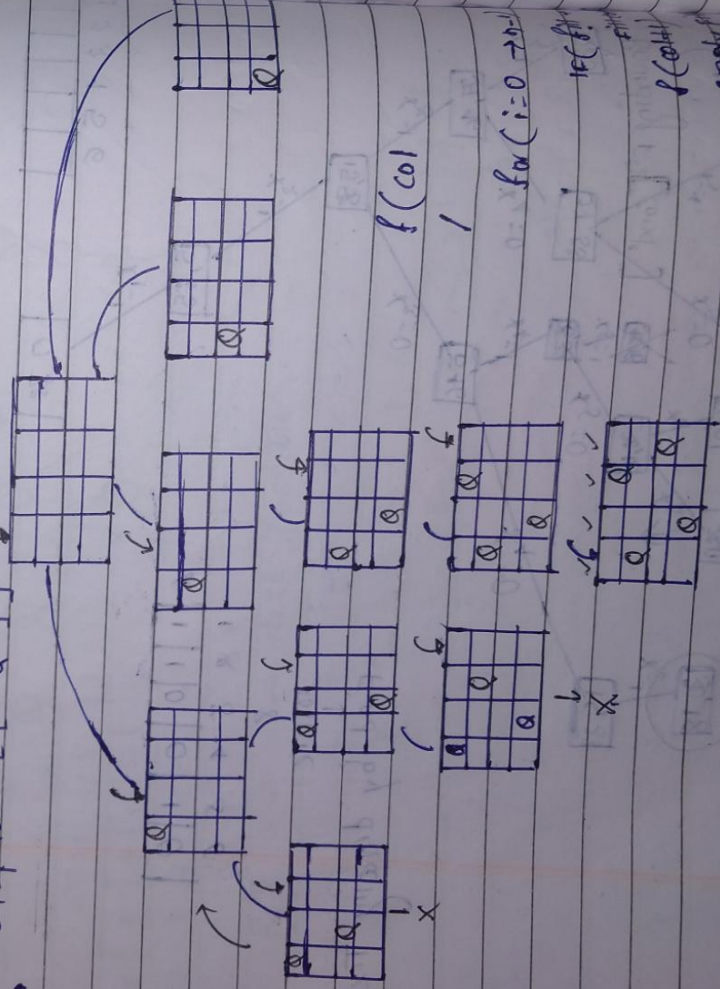
Input: n = 4

Output: ["Q...", "...Q", "...Q...", "...Q"], ["...", "...Q", "...Q...", "...Q"]

Input: n = 1

Output: ["Q"]

(going clockwise)



Public:

bool issa

that

Public:

```
void solve (int col, vector<string> &board, vector<vector<string>>&ans, int n) {
```

```
    if (col == n) {
```

```
        ans.push_back (board);
```

```
        return; } }
```

```
for (int row = 0; row < n; row++) {
```

```
    if (isSafe(row, col, board, n)) {
```

```
        board[col] = 'Q';
```

```
        solve (col+1, board, ans, n);
```

```
        board[col] = '.'; }
```

```
 }
```

Public:

```
vector<vector<string>> solveNQueens (int n) {
```

```
    vector<vector<string>> ans;
```

```
    vector<string> board(n);
```

```
    string s(n, '.');
```

```
    for (int i = 0; i < n; i++) {
```

```
        board[i] = s;
```

```
 }
```

```
    solve (0, board, ans, n);
```

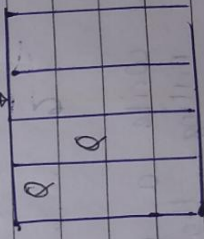
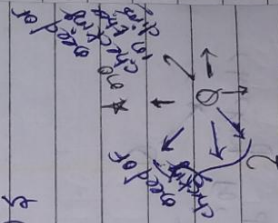
```
    return ans; }
```

Public:

```
bool isSafe (int row, int col, vector<string> board, int n) {
```

```
    int duprow = row;
```

```
    int dupcol = col;
```



while (row < 9 && col < 9) {

 // checking for this digit

 if (board[row][col] == 'Q') return false;

 row++;

 col++;

}

col = dupRow;

row = dupRow;

while (col < 9) {

 if (board[row][col] == 'Q') return false;

 col++;

 // checking for this digit

 row = dupRow;

 col = dupCol;

 while (row < 9 && col < 9) {

 if (board[row][col] == 'Q') return false;

 row++;

 col++;

 } return true;

Q. Write a program to solve a sudoku puzzle filling the empty cells.

1. Each of the digits 1-9 must occur exactly once in each column.
2. Each of the digits 1-9 must occur exactly once in each of the 9 sub-boxes of grid.

The ()

Example:-

Input:- $\begin{bmatrix} 5 & 3 & \\ & 7 & \\ 4 & & \end{bmatrix}$

Output:- $\begin{bmatrix} 5 & 3 & \\ & 7 & \\ 4 & & \end{bmatrix}$

The '.' character indicate empty cells.

Example:-

5	3			7					
6			1	9	5				
	9	8				6			
8				6			3		
4		8	8		3			1	
7				2				6	
		6				2	8		
			4	1	9				5
				8				7	9

return False

put:-
 [[5, 3, ., ., 7, ., ., ., ., .], [6, ., ., ., 9, 5, ., ., ., .]
 [., 9, 8, ., ., ., ., ., 6, .], [8, ., ., ., 6, ., ., ., 3]
 [4, ., 8, 8, ., 3, ., ., ., 1], [7, ., ., ., ., 2, ., ., ., 6]
 [., 6, ., ., ., ., 2, 8, ., .], [., ., ., ., 4, 1, 9, ., ., 5]
 [., ., ., ., ., ., 8, ., ., 7, 9]

return False

output:-
 [[5, 3, 4, 6, 7, 8, 9, 1, 2], [6, 7, 2, 1, 9, 5, 8, 4, 8]
 [6, 9, 8, 3, 2, 2, 5, 6, 7]]

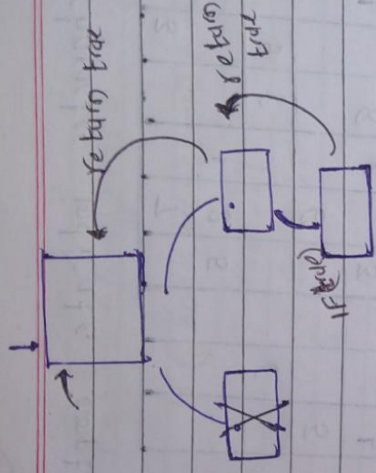
5	3	4	6	7	8	9	1	2	
6	7	2	1	9	5	3	4	8	
1	9	8	3	4	2	5	6	7	
8	5	9	7	6	1	4	2	3	
2	2	3	8	5	3	7	8	5	6
7	1	3	9	2	4	2	8	4	5
9	6	2	5	3	1	9	3	5	
2	8	7	2	8	6	1	7		
3	4	5							

puzzle by

exactly one in

exactly one

exactly one



bool

```
void solveSudoku (vector<vector<char>> & board) {
    solve(board);
}
```

```
bool solve (vector<vector<char>> & board) {
```

```
    for (int i=0; i<board.size(); i++) {
```

```
        for (int j=0; j<board[0].size(); j++) {
```

```
            if (board[i][j] == '.') {
```

```
                for (char c='1'; c<='9'; c++) {
```

```
                    if (isValid(board, i, j, c)) {
```

```
                        board[i][j] = c;
```

```
                        if (solve(board) == true)
```

```
                            return true;
```

```
                    else
```

```
                        board[i][j] = '.';
```

```
                } return false;
```

```
            } return true;
```

```
}
```

```
bool isValid (vector<vector<char>> &board, int row, int col, char c)
for (int i = 0; i < g; i++) {
```

```
    if (board[i][col] == c)
```

```
        return false;
```

```
    if (board[row][i] == c)
```

```
        return false;
```

```
    if (board (3 * (row/3) + i/3) [ 3 * (col/3) + i/3] == c)
        return false;
```

```
    return true;
```

}

++}

' ; CHDS

i, 0) {

= true)

= c. ")

12/01/21 (10/01/21) 10/01/21

For (10/01/21) 10/01/21

10/01/21 (10/01/21) 10/01/21

10/01/21 (10/01/21) 10/01/21

10/01/21 (10/01/21) 10/01/21

10/01/21 (10/01/21) 10/01/21

10/01/21 (10/01/21) 10/01/21

10/01/21 (10/01/21) 10/01/21

10/01/21 (10/01/21) 10/01/21

10/01/21 (10/01/21) 10/01/21

10/01/21 (10/01/21) 10/01/21

Handwritten text in the left margin, possibly a date or page number, appearing as "100" or "101".



DP $\rightarrow$ enhanced recursion

Identify $\rightarrow$

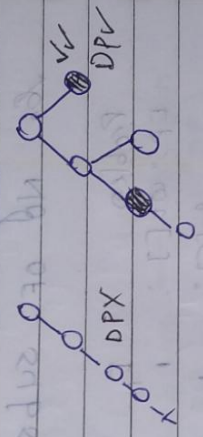
Recursion/DP

1. choice
2. Optimal

0 0 0 0

x x x x

[Recursion $\rightarrow$ calls]



Recursion $\rightarrow$ Memoize $\rightarrow$ Top-down

[Recursion] $\Leftarrow$ Must

Memoization

Top-down

subset sum
equal sum partition
 $\rightarrow$ count of subsets

1. 0-1 Knapsack (6)
2. Unbounded Knapsack (5)
3. Fibonacci (7)
4. LCS (15)
5. LIS (10)
6. Kadane's Algorithm (6)
7. Matrix chain multiplication (7) loop
8. DP on Trees (4)
9. DP on Grid (4) $\sim$ matrix
10. Others (3)

~~Greedy~~

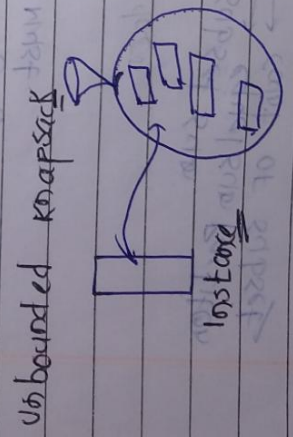
Types of Knapsack $\rightarrow$ Fractional Knapsack $\rightarrow$ Unbounded Knapsack

0-1 Knapsack problem

1. Subset sum
2. Equal sum partition
3. Count of subset sum
4. Minimum subset sum diff
5. Target sum
6. No. of subset of given diff

Problem $\begin{matrix} I_1 & I_2 & I_3 & I_4 \\ \text{IP: wt}[] : & 1 & 3 & 4 & 5 \\ \text{val}[] : & 1 & 4 & 5 & 7 \end{matrix}$

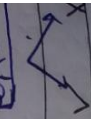
$N = 7$
capacities



unbounded knapsack

choice di

$\text{if } x = 0$



~~Greedy~~
Knapsack

0/1 Knapsack

Identify $wt[]$: 1 3 4 5
 $val[]$: 1 4 5 7
 $w: 7kg$ o/p: Max Profit
1 choices x ✓ x ✓ x ✓ → choices

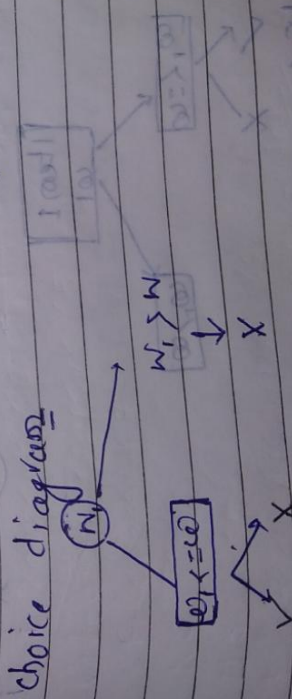
2. Optimal

DP

DP: → Recursion → Memorization → Top-down
sop ($0 = 0$ || 0 (DP)) (PP)

Recursion
DP = (q) + (v)
storage

Recursive Code
↓
choice diagram



max Profit

↑
list Knapsack (int wt[], int val[], int w, int n)

Base Cond.

Choice Diagram

Base Condⁿ → Think of the smallest valid IP



IF (n == 0 || w == 0)

return 0

Recursive Fun: Fun(IP)

IP -> Fun (smaller IP)



Item 1

wt, val

wt, val

IF (wt[n-1] <= w)

return val[n-1] + Knapsack(wt, val, n-1, w-1)

else

return Knapsack(wt, val, n, w)

Memorization

$$\boxed{\omega_1 > \omega_2}$$
$$IF(wt[h-1] \leq w)$$

return Knapsack(wt, val, w, d-1)

```
int knapsack(int wt[], int val[], int w, int n)
```

3

$$f(x) = 0 \quad || \quad 0 = 0$$

return 0

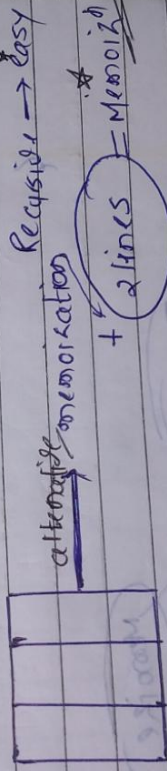
$$F(\omega) \in [0, 1] \quad \forall \omega$$

```
return max (val [i-1] + knapsack (wt, val, w, n-1),
            knapsack (wt, val, w, n-1));
```

$$\text{else if } (wf(n-1) > w)$$

return Knapsack(wt, val, w, n-1);

Memorization



Top - down

Recursive	$\Rightarrow$	DP
+		
2 hrs		

Kompact (wt[], wt[]), wt[n+1]

matrix of changing variable

we make
inf + [on +] [of +]

$$\text{Set}(t_{i-1}, \text{sizeof}(t))^{n+1}$$

static

* int t[102][1000]
memset(t, -1, sizeof(t))

0/1 Knapsack

int Knapsack(int wt[], int val[], int w, int n)

{ if (n == 0 || w == 0)

return 0

if (t[n][w] != -1)

return t[n][w]

if (wt[n] <= w)

return max(val[n] + Knapsack(wt, val, w - wt[n], n - 1), Knapsack(wt, val, w, n - 1))

else if (wt[n] > w)

return Knapsack(wt, val, w, n - 1)

wt[] =
val[] =

wt	1	3	4	5
val	4	5	7	8

* match with one box

except scramble string

Memorize

Top-down

* Both are O(n²)

Both Passes all Test Cases

Subproblems

subproblems to solve

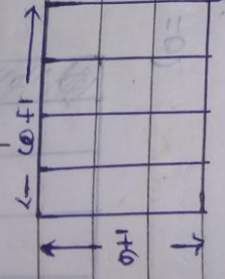
val	4	5	7	8
wt	1	3	4	5

n, w

$n < 100$
 $w \leq 100$

w, int

Knapsack Top-down approach



step 1 initialize

step 2 - Fun Recursive call

→ initialize iterative

$wt[] = [0, 1, 3, 4, 5]$

$n = 4$

$val[] = [1, 4, 5, 7, 10]$

$wt + [1, w = 7]$

$t[wt+1][wt+1] \rightarrow t[5][8]$



scrambled string

$O(n^2)$

match with one back

Max Profit

$wt[] = 1, 3$
 $val[] = 1, 4$
 $w = 6$

$wt[] = 1, 3, 4$
 $val[] = 1, 4, 5$
 $w = 6$

subproblems

$wt[] = 1, 3, 4, 5$
 $val[] = 1, 4, 5, 7$
 $w = 7$

In recursive

* Base condⁿ



initialization



IF (n == 0 || m == 0)

return 0;

Recursive

Top-down

for (i = 0; i < n+1; i++)

for (j = 0; j < m+1; j++)

if (i == 0 || j == 0)

t[i][j] = 0

if (m < n-1 < m)

return max(

+ knapsack

(

t[n-1][j] <= m)

t[n][j] = max(

val[n-1][j] + t[n-1][j-1]

t[n][j-1])

else if

(m < n-1 > m)

return knapsack

(m, val, m-1)

for (int i = 1; i < n+1; i++)

for (int j = 1; j < m+1; j++)

{

if (m < i-1 <= j)

t[i][j] = max(

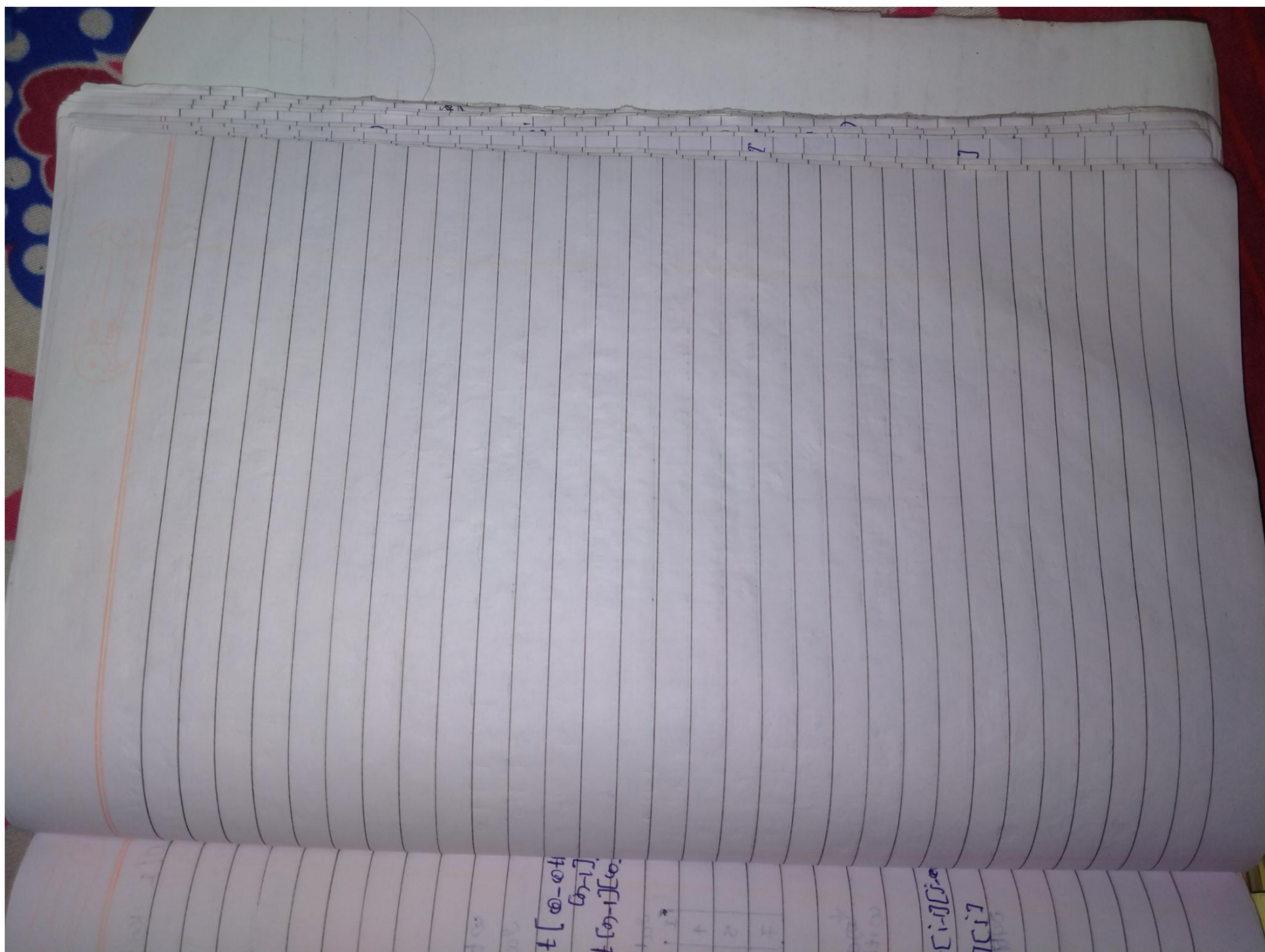
val[i-1][j] + t[i-1][j-1]

t[i-1][j])

else

t[i][j] = t[i-1][j]

return t[n][m];



subset sum problem

Base

IF (sum == 0)
return true;
IF (n == 0)
return false
IF (arr[n-1] < sum)
return is(arr, n-1, sum)
IF is(arr, n-1, sum - arr[n-1])
else
return is(arr, n-1, sum)

arr[]: 2 3 7 8 10
sum = 11

2 3 7 8 10
✓ x ✓ x ✓ x ✓ x ✓ x

you have to find the difference

t[n+1][sum+1] t[n+1][0] t[0][sum+1]

	0	1	2	3	4	5	6	7	8	9	10	11
0	T	F	F	F	F	F	F	F	F	F	F	F
1	T											
2	T											
3	T											
4	T											
5	T											

O/P → True / False

KnapSack

subset sum

IF (wt[i-1] <= j)

IF (arr[i] <= i)

t[i][j] = max(val[i-1] + t[i-1][j-wt[i-1]], t[i-1][j])

|| t[i-1][j]

else

t[i][j] = t[i-1][j]

t[i][j] = t[i-1][j]

Max(T, F)

[True]

return (b)[sum]

* Initialising vector of size (NH) (sum+1)

vector <vector<int>> t(NH, vector<int>(sum+1));

2. Equal sum partition

arr[]: {1, 5, 11, 5, 6}

O/P: T/F

{ 6, 3, 6 } → { 1, 5, 5, 6 } { 11, 6 }

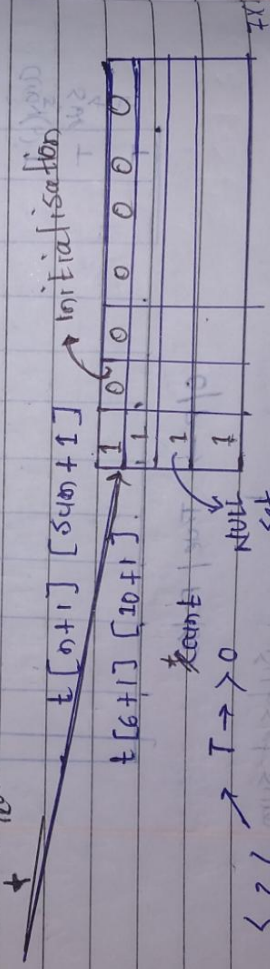
586 are based on this

3. Count of subset sum

arr[]: {2, 3, 5, 6, 8, 10}

sum: 10

{ 2, 8 } { 5, 2, 3 } { 10 } O/P → 3



{ 2, 6 } → T → > 0
F → 0

IF (arr[i-1] <= j)

t[i][j] = t[i-1][j] + t[i-1][j-arr[i-1]]

else

Recursive t[i][j] = t[i-1][j]

IF (sum == 0)

return 1

IF (n == 0)

return 0

return t[i][j]

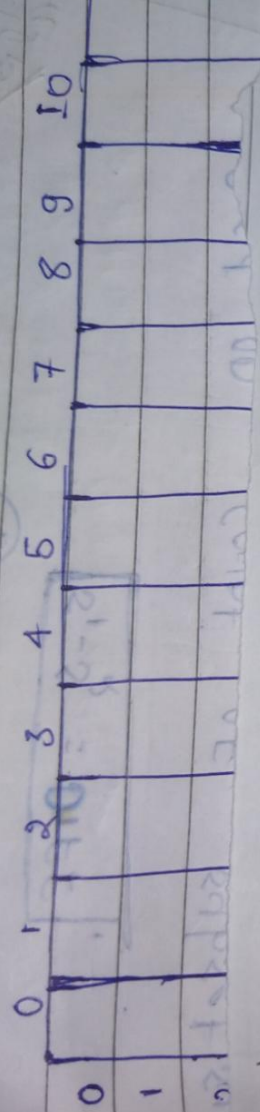
return t[i][j]

return t[i][j]

return t[i][j]

b/w 'arr' =

arr: [1, 2, 7]



int traverse(int arr[], int n, int sum, int total, tab)

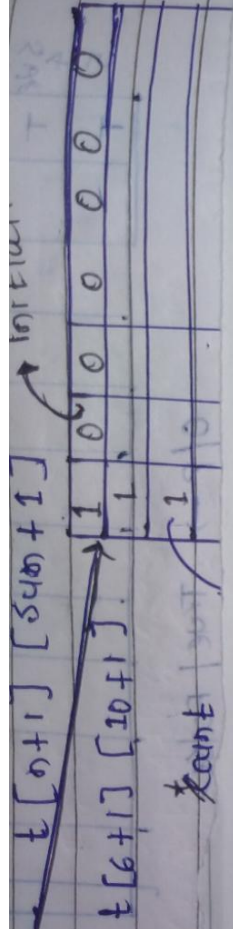
{ if (tab[sum] [n-1]) { return tab[sum]; }

if (n <= 0)

{ return tab[sum] [sum] = abs(tot - sum); }

else

{ return tab[sum] [sum] = min(traverse(arr, n-1, sum+arr[n-1], tot, tab), traverse(arr, n-1, sum, tot, tab)) }



int main()

{ int total = 0, s4n = 0;

for (int i = 0; i < n; i++)

{ total += arr[i];

}

return total < int> tab(n+1, s4n+1) (total+1);

return traverse(arr, n, s4n, total, tab)

}

5. Count the subset with sum equal to x
 $\rightarrow arr[] = \{1, 2, 3, 3\}, x = 6$

O/P = 3

Possible subsets are $\{1, 2, 3\}$ and $\{3, 3\}$

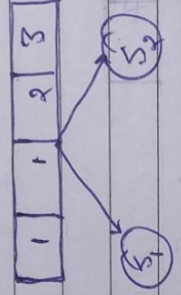
$arr[] = \{1, 1, 1, 1\}, x = 1$

O/P = 4

5. Count the subset with given difference

$arr[] = \{1, 1, 2, 3\}$

DIFF: 1



$S_1 - S_2 = \text{DIFF}$

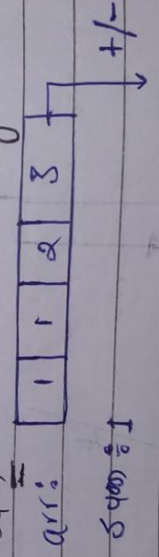
Goal is to

Based on Count of subset sum

Do it For relation $(3, 4, 5)$ will be result

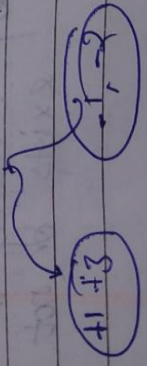
6. Target sum =

Using just with single element



Sum = 1

$+1 - 1 - 2 + 3 = 1$



S_1 S_2

$S_1 - S_2 = \text{DIFF}$ based

on previous problem

always remember

Specialty for starting from 0

Constraints

$0 \leq n \leq 1000$

```
int FindTargetSubways (vector<int> & nums, int target)
    int n = nums.size();
```

```
    int sum = accumulate (nums.begin(), nums.end(), 0);
    int new-target = (sum+target)/2
```

```
    if (sum < target || (sum+target)%2 != 0) return 0;
```

```
    int dp[n+1][new-target+1];
```

```
    for (int j=0; j<new-target+1; j++) dp[0][j] = 0;
```

```
    dp[0][0] = 1;
```

```
    for (int i=1; i< n+1; i++)
```

```
        for (int j=0; j<new-target+1; j++) {
```

```
            if (nums[i] <= j)
```

```
                dp[i][j] = dp[i-1][j-nums[i]] + dp[i-1][j];
```

```
            else
                dp[i][j] = dp[i-1][j];
```

```
        return dp[n][new-target];
```

Your input: [0,0,0,0,0,0,0]

target = 1

Ans:- 256

Constraints:

0 <= nums[i] <= 1000

OR pow(2, no-of-zero) * f()

if solving recursively

always

unbounded

Minimum subset sum difference

another approach:- (using Memoization)

```
int traverse (int arr[], int n, int sum, int total,
              vector <vector<int>> &tab)
```

```
    if (tab[n][sum] != -1) { return tab[n][sum]; }
```

```
    if (n <= 0)
```

```
    {
```

```
        return tab[n][sum] = abs (total - sum);
```

```
    } else
```

```
    { return tab[n][sum] = min (traverse (arr, n-1, sum, total, tab),
```

```
                                traverse (arr, n, sum, total, tab)); }
```

```
    }
```

```
int minDifference (int arr[], int n) {
```

```
    int total = 0, sum = 0;
```

```
    for (int i = 0; i < n; i++)
```

```
    { total += arr[i]; }
```

```
    }
```

```
    vector <vector<int>> tab (n+1, vector<int> (total+1, -1));
```

```
    return traverse (arr, n, sum, total, tab); }
```

```
if (0 <= i <= j)
    dp[i][j] = max (dp[i-1][j],
                    dp[i][j-1]);
else
    dp[i][j] = dp[i][j-1] + dp[i-1][j];
```

01 Knapsack

0	0	0	0	0	0
---	---	---	---	---	---

✓

✓

01

0	0
---	---

0	0	0	0	0
0				
0				
0				

bounded knapsack =

Rod cutting

coin change I

coin change II

Maximum Ribbon cut not found

01 Knapsack

Unbounded knapsack

tab[0][0] = 0, val[0] = 0, wt[0] = 0

✓

✓ X

✓ X

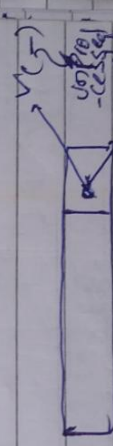
After that

we don't comp here

Multiple occurrence of same

item → allowed

Processed



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

processed (not come again)

if (wt[i-1] <= j)

tab[i][j] = max(val[i-1] + tab[i-1][j-wt[i-1]], tab[i-1][j])

else

tab[i][j] = tab[i-1][j]

if (wt[i-1] <= j)

tab[i][j] = max(val[i-1] + tab[i-1][j-wt[i-1]], tab[i-1][j])

else

tab[i][j] = tab[i-1][j]

1. Rod cutting problem

length[]: [1, 2, 3, 4, 5, 6, 7, 8]
 price[]: [1, 5, 8, 9, 10, 17, 19, 20]

$n=8$

Same as unbounded Knapsack

2. Coin change problem

Based on count of subset problem

3. Coin change problem [Min^m number of coins]

coins[] = {25, 10, 5, 1, 2}, $V=30$

Min^m coins required

Min^m coins required

Min^m coins required

coins	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Min ^m coins	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

Min^m coins required

Min^m coins required

Min^m coins required

Min^m coins required

Min^m coins required

Min^m coins required

if (

len[]

else

return

len[]

return

len[]

return

len[]

return

len[]

return

len[]

return

len[]

return

len[]

return

len[]

return

len[]

return

len[]

return

if (length[i-1] < i)

dp[i] = max(dp[i-1] + t[i], wt[i-1])

else

dp[i] = t[i] + dp[i-1]

0. N = t, x = 2, y = 1, z = 1

traverse(int den, vector<int> x, y)

for (int i = 0; i < x; i++) { dp[i][0] = 0; }

for (int j = 1; j < den; j++) { dp[0][j] = INT_MAX; }

for (int i = 1; i < x; i++) {

for (int j = 1; j < den; j++) {

dp[i][j] = max(dp[i-1][j],

dp[i-1][j-den] + t[i]);

return dp[x-1][den];

traverse(den, x, y-1);

return dp[x-1][den];

int dp[n+1][amount+1];

for (int i = 0; i < n; i++) { dp[i][0] = 0; }

for (int i = 0; i < amount; i++) {

for (int j = 0; j < n; j++) {

dp[j][i] = min(dp[j][i],

dp[j-1][i-coins[j]] + coins[j]);

return dp[n-1][amount];

dp[j][i] = dp[j-1][i];

dp[j][i] = dp[j-1][i];

dp[j][i] = dp[j-1][i];

dp[j][i] = dp[j-1][i];

dp[j][i] = dp[j-1][i];

dp[j][i] = dp[j-1][i];

dp[j][i] = dp[j-1][i];

dp[j][i] = dp[j-1][i];

dp[j][i] = dp[j-1][i];

Note: [★] Do all these Questions

Longest Common Subsequence

- 1) Longest Common Substring
- 2) Print LCS (seen logic)
- 3) Shortest Common Supersequence
- 4) Print LCS $\rightarrow$ based on edit distance
- 5) Min. # of insertion and Deletions $a \rightarrow b$
- 6) Largest Repeating Subsequence
- 7) Largest of Largest Subsequence of a where is a sub
- 8) Subsequence pattern Matching
- 9) Count How many times a letter as subsequence
- 10) Largest Palindromic Subsequence
- 11) Largest Palindromic Substring
- 12) Count of Palindromic Substring
- 13) Min. # of Deletion in a string to make it a pal
- 14) Min. # of insertion in a string to make it

Palindromic

Flow $\rightarrow$ Recursive
bottom up
LCS
Memorization
Top-down

15 Problems

another Q
ex - Max

Longest

Input: S1 = "A"
O/P: 4

void traverse (int len)
if (len == 0 || 1)
return;

else if (S1[len-1] ==
if (max < len)
max = len;
traverse (len-1);

else
traverse (0, len)
traverse (len, 0);

Longest P
I/P: S = "bbbab"
O/P: 4

One Possible LPS is
 $\rightarrow$ string S1 = S

$$g_{\text{KMS}} = [1, 2, 3, 2, 1]$$

Q → longest Palindromic substring

1R" 10P44: S1 = "ABC", S2 = "ACB"
O/P: 1

~~1P1~~ string (strings, int)

$$[1+G] \frac{dP}{dP} = 1$$
$$\frac{1}{10} = 0.1$$
$$0 \left(\text{ind } j = 0, \quad j \times \text{ind } j = 0 \right)$$
$$\dots \quad i=1^n$$

3

$$(\mathbb{F}_q[x])^* = (\mathbb{F}_q[x])^*$$

$(H_1^1)^2 \rightarrow H_1^1 \rightarrow H_1^1$

ence

6
p q q p

possible dps is $aabb$.

$\text{pos}(\text{ss}(\text{begin}(\text{ss}(\text{end})))$

→

100 24 100 20 100 15 100 10 100 5 100 0

Q. Longest Repeated subsequence

Input: str = "ABEBCDP"

o/p: "ABD"

Print longest repeating subsequence so that two subsequence don't have same string character at same pos.

Same as LCS, only change we have to do is

if (str[i-1] == str[j-1] && i != j)

dp[i][j] = 1 + dp[i-1][j-1]

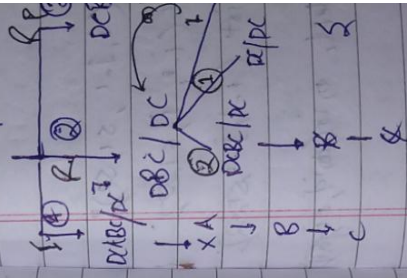
Q. T

Edit Dist

s1 = abc

s2 = bcd

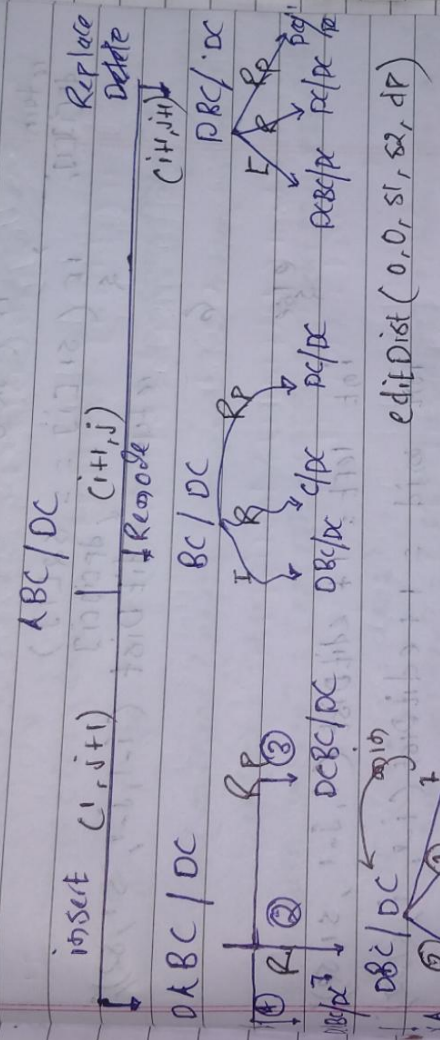
insert c, j+1
DABC/DC



insert
replace
delete

Edit Distance

$s1 = abc \Rightarrow 2$
 $s2 = bcd \Rightarrow 3$
 $s1 = \text{Sunday} \Rightarrow 8 \text{ steps}$
 $s2 = \text{Saturday} \Rightarrow 8 \text{ steps}$



IF $(i = s1.size())$ return $s2.size() - i$
 IF $(j = s2.size())$ return $s1.size() - j$
 IF $(dp[i][j] != -1)$ return $dp[i][j]$
 IF $(s1[i] == s2[j])$ return $1 + \text{editDist}(i+1, j+1, s1, s2, dp)$
 IF $(s1[i] != s2[j])$ return $1 + \min(\text{insert}, \text{delete}, \text{replace})$

insert $\leftarrow 1 + \text{editDist}(i, j+1, s1, s2, dp)$
 delete $\leftarrow 1 + \text{editDist}(i+1, j, s1, s2, dp)$
 replace $\leftarrow 1 + \text{editDist}(i+1, j+1, s1, s2, dp)$
 return $dp[i][j] = \min(\text{insert}, \text{delete}, \text{replace})$

↓ going from back side

int editDist(

if (i < 0) return j + 1;

if (i < 0) return i + 1;

if (s1[i] == s2[j])

return editDist(i-1, j-1, s1, s2);

return 1 + editDist(i-1, j, s1, s2);

else

int left = 1 + editDist(i, j-1, s1, s2);

int mid = 1 + editDist(i-1, j, s1, s2);

int right = 1 + editDist(i-1, j-1, s1, s2);

return dp[i][j] = min(left, mid, right);

Q. Interleaved
Ex: a a b

a

bool check

string k

if (

return

if (

return

bool

if (s1

return

if (s2

return

Interleaving string
ex: a a b c d A b b c a
 ↓ ↓ ↓ ↓ ↓
 a a b b c c d d

bool check (i1, P2, P3, s1, s2, s3, len1, len2, len3)

if (P3 == len3) & return true;

string Key = to-string(P1) + "*" + to-string(P2) + "*" + to-string(P3);
if (indexOf(Key) != -1) & return Key;

if (P1 == len1)
& return Key = (s2[P2] == s3[P3]) ? check(P2+1, P3+1, len2, len3) :

if (P2 == len2)

& return Key[P1] = (s1[P1] == s2[P2]) ? check(P1+1, P2+1, len1, len2) :

bool left = false; right = false;

if (s1[P1] == s2[P2])

& left = check(P1+1, P2, P3+1, s1, s2, s3, len1, len2, len3)

if (s2[P2] == s3[P3])

& right = check(P1, P2+1, P3+1, s1, s2, s3, len1, len2, len3)

& return Key = (left || right);

Q. Print SCS

a: "geek" $\Rightarrow$ geek
b: "geek" $\Rightarrow$ A G G T A B T X A Y B

i = 0, j = 0

```
while(i < 0 && j < 0)
{
    if(str1[i-1] == str2[j-1])
    {
        rest = str1[i-1];
        i--; j--;
        while(j < 0)
        {
            if(tab[i-1][j] > tab[i][j-1])
            {
                rest = str1[i-1];
                i--;
            }
            else
            {
                rest = str2[j-1];
                j--;
            }
        }
    }
}
```

return rest;

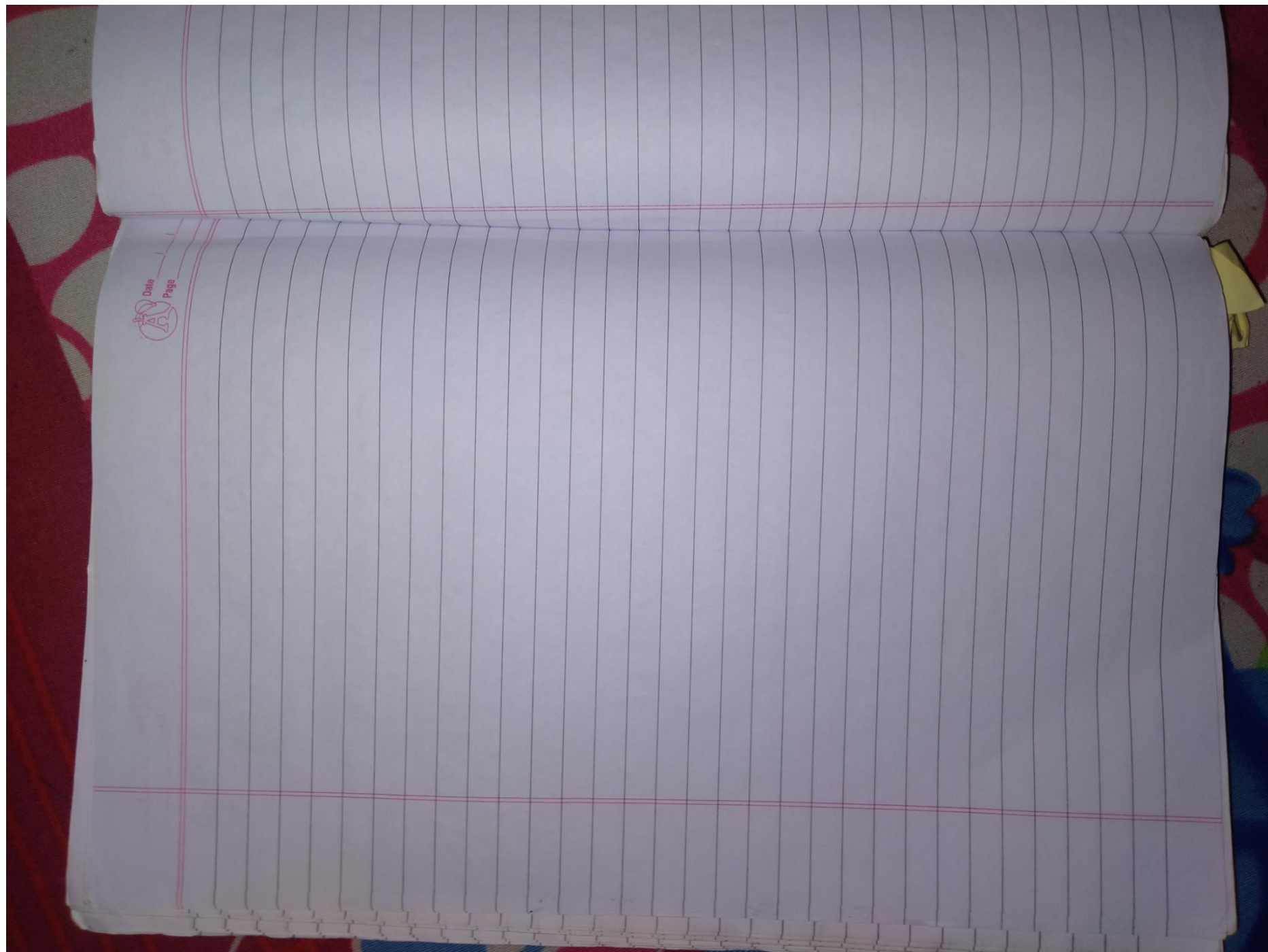
1000

7G x 1A 1B
1 x B

Fig-

2 [0-1]

1000



st
st

Largest common
 recursive

I/P x: abc

y: abcd

x: (a) (b)

y: (a) (b)

Largest common sub

1. Base cond

y: c

y: c

IFC

Largest common subsequence / LCS $\rightarrow$ it may be discontinuous

179 x: a b c d g h

$$Y: abedfha$$

31 x 60

strings;

$c_1 \mu_1 \mu_2 \mu_3$

8/25/2

x: a b c d f h
y: a b e d f b a

2: a b e d f a

→ $\boxed{abcdh} \rightarrow 4$ Ans

most common subtyping it shouldn't be discontinuous

96P-9D

$\square a$ $\square b$ $\square c$ $\square d$ $\square e$ $\square f$ $\square g$ $\square h$ $\square i$ $\square j$ $\square k$ $\square l$ $\square m$ $\square n$ $\square o$ $\square p$ $\square q$ $\square r$ $\square s$ $\square t$ $\square u$ $\square v$ $\square w$ $\square x$ $\square y$ $\square z$

ab → substring

1. Base condⁿ :- Think of smallest + valid I/P

A ditya $\rightarrow 6$ - 500000

ferend $\rightarrow$ 5 = small

γ :

0

$\downarrow$
 $b = 0, a = 0$

$$IF(\eta = 0 \parallel \eta = 0)$$

2. choice diagram

x: abc dgh

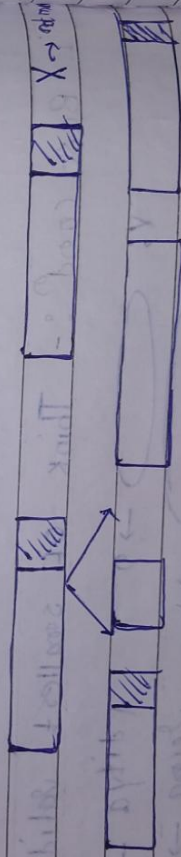
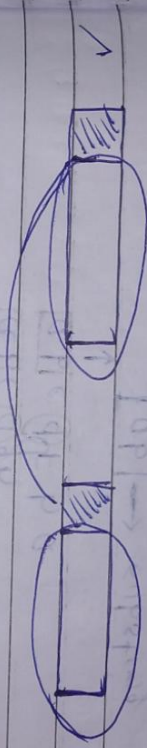
y: abedfhr

x: abcdgh
y: abedfhr

→ h

IF $x[0-1] == y[0-1]$ IF q_{11}

x: abcdgh or abedfhr
y: abedfhr



IF $(0 == x || 0 == y)$

int LCS (string x, string y, int n, int m)

{
 Base Condition → IF (n == 0 || m == 0)

 return 0;

choice:

IF (x[n-1] == y[m-1])

 return 1 + LCS (x, y, n-1, m-1)

else

 return max (LCS (x, y, n, m-1) ,
 LCS (x, y, n-1, m)) ;

LCS Memoized (Bottom Up)

```
static  
int t[1001][1001];
```

```
int main()
```

```
{  
    memset(t, -1, sizeof(t));
```

```
    int LCS()
```

```
{  
    memset(t, -1, sizeof(t));
```

```
    if (n == 0 || m == 0)
```

```
        return 0;
```

```
    if (t[n][m] != -1)
```

```
        return t[n][m];
```

```
    if (x[n-1] == y[m-1])
```

```
        return
```

```
        t[n][m] = 1 + LCS(x, y, n-1, m-1);
```

```
    else
```

```
{
```

```
        return
```

```
        t[n][m] = max(LCS(x, y, n, m-1),
```

```
    }
```

DP - LCS

$X: a b c d$
 $Y: a b c d e$

$\rightarrow abcde$

if $(X[h-1] == Y[h-1])$

$t[h][h] = 1 + t[h-1][h-1]$

else

$t[h][h] = \max(t[h][h-1], t[h-1][h]) + 1$

100%

100%

① Longest common substring

sr a: abcde

or: 2

b: abfce

→

a: a b e

b: a b f

if (t[i-1] == t[j-1])

t[i][j] = t[i-1][j-1] + 1 ;

else

t[i][j] = 0 ;

② Longest repeating subsequence

str = "A A B C B C D D"

A A B C B C D D

→ ABD

③ Printing Longest common subsequence

→

a: abcde

b: abcdef

o.p:- "abcde"

a: a c b c f $\rightarrow$ h[5]
 b: a b c d a f $\rightarrow$ n[6]

LCS $\rightarrow$ t[m+1][n+1] $\rightarrow$ t[6][7]

	a	b	c	d	a	f
0	0	0	0	0	0	0
a	1	1	2	2	2	1
b	0	1	1	2	2	2
c	0	0	1	2	2	2
d	0	0	1	2	3	2
f	0	0	1	2	3	3

$\rightarrow$ "f c b a"

if (a[i-1] == b[j-1])

t[i-1][j-1] + 1

else

max(t[i-1][j], t[i][j-1])

Print LCS

return i, j, max

while (i > 0 && j > 0)

if (a[i-1] == b[j-1])

s.push_back(a[i-1]);

j--;

i--;

else

j--;

i--;

if (t[i-1][j-1] > t[i][j-1])

j--;

i--;

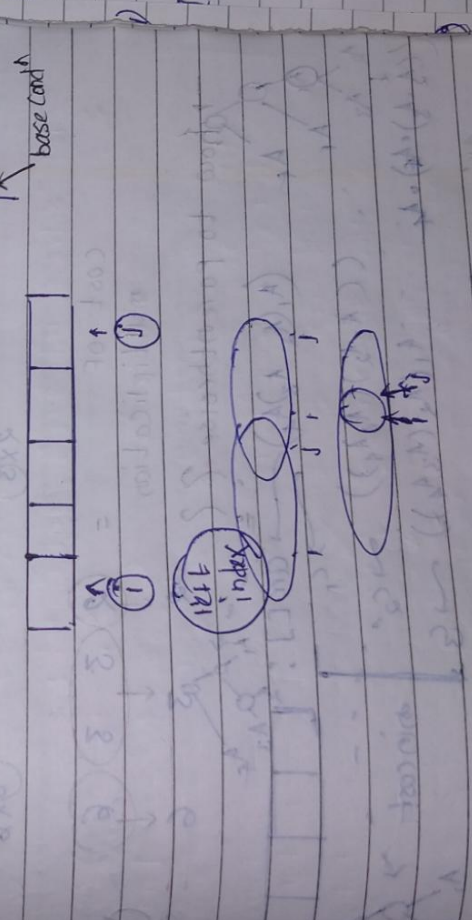
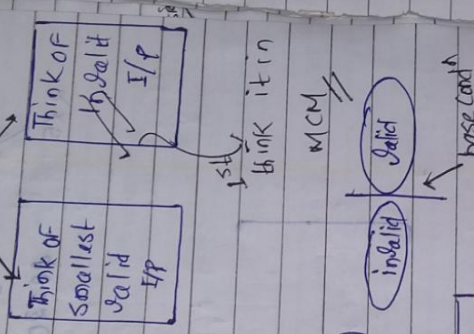
j--;

i--;

→ Confusing (we will see)
 Recursive
 → Memo
 → True/Boolean
 value of an
 partitioning
 problem

~~code~~
 int solve (int arr[], int i, int j)
 IF (i > j) → diff according to Q
 return 0
 (int k = i; k < j; k++)

// calculate temp ans
 tempAns = solve(arr, i, k)
 + solve(arr, k+1, j)
 ans ← fun(tempAns)
 Max/Min ... (according to Q)



by temp ans

Flow
understand → Problem statement
deeply

Recursive

→ Code using the Form

easy

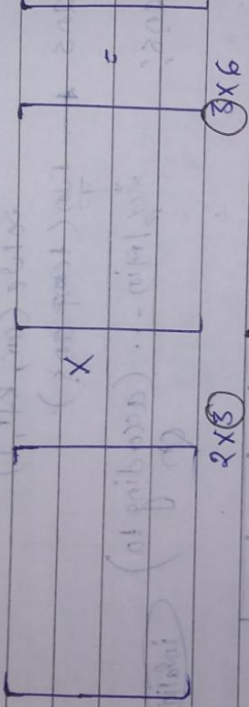
arr[] = { 5 40 20 30 10 30 }

A₁ A₂ A₃ A₄

2x5 40x30 30x10 10x30

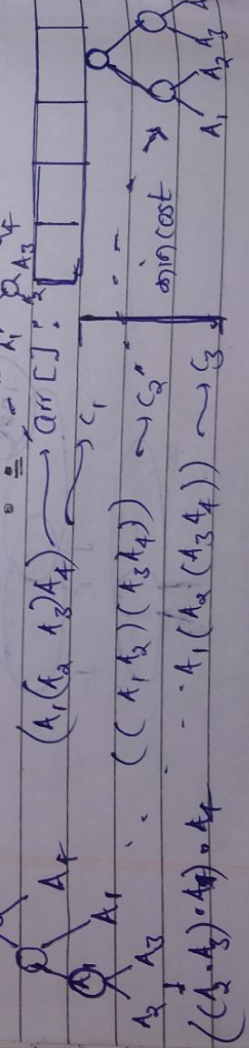
A₁ A₂ A₃ A₄ cost → mid

no of multiplication



Cost of multiplication = 2 3 3 6 6 6 = 36 cost

→ how to parenthesize??



ex:- A → 10x30 (A₁)C → 10x30 30x5 =

B → 30x5 A(B)C → 10x30 30x5 =

C → 5x60 A(B)C → 10x30 30x5 =

cost → 10x30x5 + 10x5x60

= 450

A(B.C) →

50x5x60 + 10x30x60

arr[] :

matrix

next step → Form

(A₁ A₂) (A₃ A₄)

(A₁)

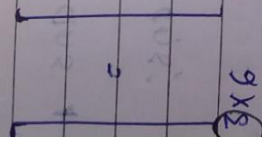
Statistics

using the Form 1

easy

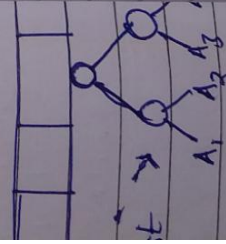
Mr D

= multiplication



Q66F and 111F

$$6 - 36 \text{ (of)}$$



$$\Rightarrow 10 \times 30 \quad 30 \times 5 =$$

$$10 \times 5 = 50$$

10x30x5 +

10x5x60

$$= 45 \text{ W}$$

$A(B.C) \rightarrow 30 \times 5 \times 60$

30×60

$$10 \times 30 \quad 30 \times 60$$

$10 \times 30 \times 60$ + 2700 $\rightarrow$ Temp Δ

Out 2 → 615.00
5/1/05

0 arr [] : 400 20 [-30] 10 [-30]

→ 4 matrix $[0]_{1 \times 1} + [1]_{1 \times 1} \rightarrow$

$$F_1 \rightarrow 40 \times 20$$

$$A \rightarrow 20 \times 30$$

$$A \rightarrow 30 \times 10$$

3 $A \rightarrow 10 \times 30$

17509 077

next step $\rightarrow$ Format



$$(A_1, A_2)(A_3, A_4)$$

Min, Min $\rightarrow$ Temp ans

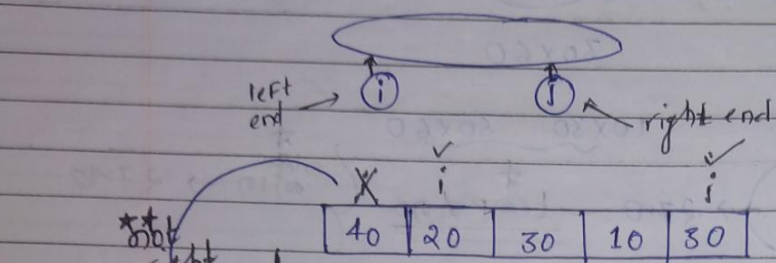
$(A_1, A_2, A_3) \in (A_2)$

min temp

[Faint, illegible handwriting visible through the paper]

This image shows a blank sheet of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page. There is no handwriting or other markings on the paper.

Min (Temp Ans)



$$\begin{aligned}
 A_i &\rightarrow arr[i-1] * arr[i] \\
 &\rightarrow arr[i] * arr[0] \\
 A_j &\rightarrow arr[j-1] * arr[j] \\
 &\rightarrow arr[3] * arr[4]
 \end{aligned}$$

First step $\boxed{i = 1} \quad \boxed{j = 4-1}$

```

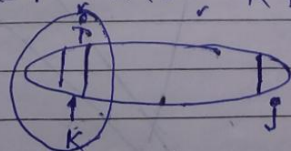
int solve ( int arr[], int i, int j )
{
    if ( i > j ) return 0;
    if ( i == j ) return arr[i-1] * arr[i];
}

```

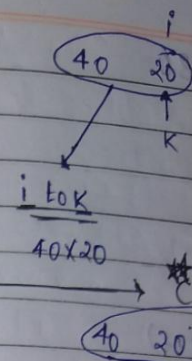
if (i > j) → return 0
*if (i == j) → it is also arr[i-1] * arr[i]*

- Step
1. Find i & j value ✓ found
 2. Find right B.C (Base Cond) ✓
 3. Move K → i to j

For (int K = i ; K < j ; K++)



how to break ??
 solve (i → K)
 solve (K+1 → j)



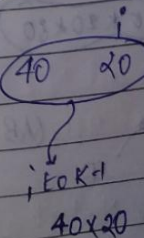
40 20

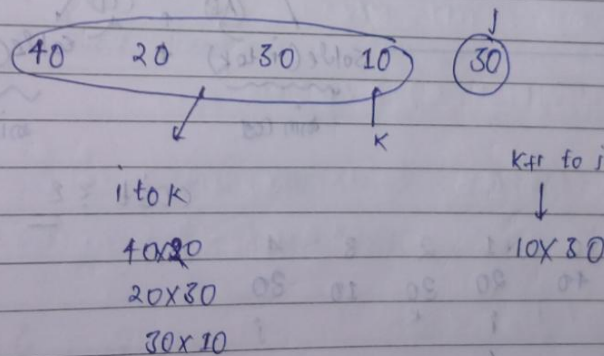
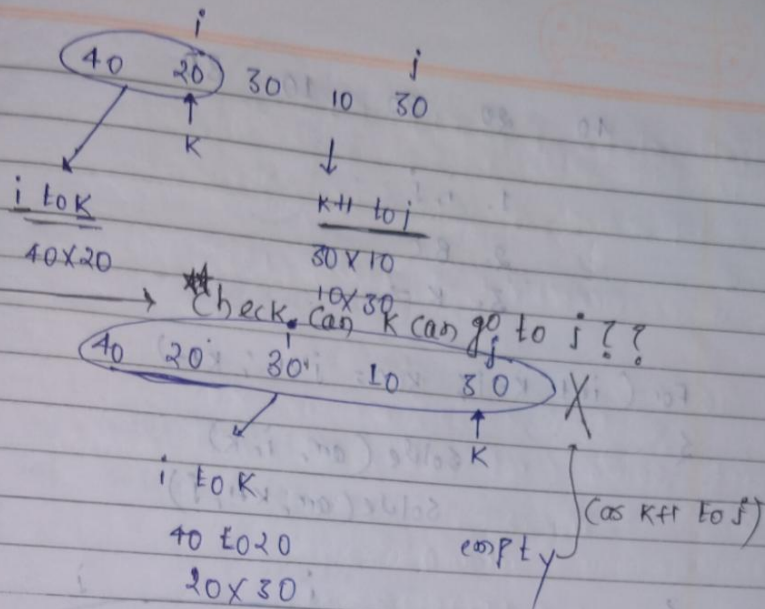
choose 1

→ K = i

choose 2 K = i+1

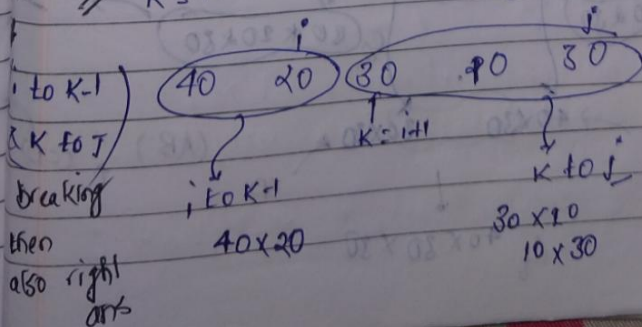
to K-1
 K to j
 breaking
 then
 also right arr





base case 1
 $\rightarrow K = i \quad K = j - 1 \quad (i \text{ to } K) \quad (K+1 \text{ to } j)$

base case 2
 $K = i + 1 \quad K = j \quad (i \text{ to } K-1) \quad (K \text{ to } j) \text{ breaking}$



40 20 30 10 30

1. i, j

2. BC

3. $K \rightarrow$

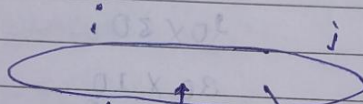
For (int $k=i$; $k \leq j-1$; $k++$)

{

Solve(arr, i, k)

Solve(arr, k+1, j)

}



Temp
ans

Solve(i to k)

min cost

+ Solve(k+1 to j)

min cost

COST ? 2

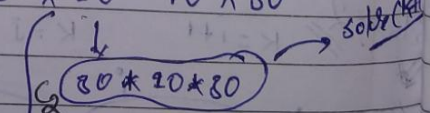
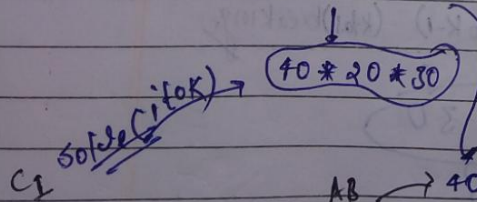
0 1 2 3 4
40 20 30 10 30
i k j

i to k

k+1 to j

40x20 20x30

30x10 10x30



AB

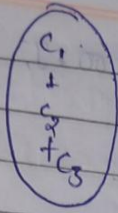
30x30

CP

(AB)

(CD)

C3 40x30x30



tempAns

40 x 80 x 30

$arr[i+1] * arr[k] * arr[j]$

(AB) (CD)

int solve (int arr[], int i, int j)

if (i == j)

return 0;

for (int k = i; k <= j-1; k++)

int tempAns = solve(arr, i, k) + solve(arr, k+1, j)

+ arr[i+1] * arr[k] * arr[j];

if (temp < min)

min = temp;

}

return min;

Rec(k+1 to j)

min cost

30

solve(k)

AB)

(CD)

```
int static t [100] [100];
```

```
int main()
```

```
{
```

```
memset (t, -1, sizeof(t));
```

```
OR
```

```
int solve (int arr[], int i, int j)
```

```
{
```

```
if (i == j)
```

```
return 0
```

```
if (t[i][j] != -1)
```

```
return t[i][j]
```

```
int min = INT_MAX;
```

```
for (int k = 1; k < j; k++)
```

```
{
```

```
int tempans = solve(arr, i, k) + solve(arr, k+1, j)
```

```
+ arr[k-1] * arr[k] * arr[j]
```

```
if (temp < min)
```

```
{
```

```
min = temp;
```

```
}
```

```
}
```

```
return t[i][j] = min;
```

```
}
```

```
int matrixmultiplication (int N, int arr[])
```

```
{  
vector<vector<int>> tab (N+1, vector<int>(N+1));
```

```
return solve (arr, 1, N-1, tab);
```

```
}
```

Palindrome part

5: nitin

nitin ~ minin

int solve (string

if (i == j)

if (is palind

int min

if (tab[i][j]

for (int k =

{

int temp

if (temp

min =

tab[i][j]

return min;

}

Palindrome partitioning

S: nitik

nitik → minimize no. of partitions nitik

int solve (strings, int i, int j)

IF (i == j) return 0;

IF (isPalindrome (s, i, j) == true) & return 0;

int min = INT_MAX;

IF (tab[i][j] != -1) & return tab[i][j];

For (int k = i; k <= j-1; k++)

{

int temp = 1 + solve (s, i, k) + solve (s, k+1, j)

IF (temp < min)

min = temp;

tab[i][j] =

return min;

}

Optimized

For (int k = i; k <= j-1; k++)

{

IF (tab[i][k] != -1)

left = tab[i][k];

else

left = solve (s, i, k)

tab[i][k] = left

IF (tab[k+1][j] != -1)

right = tab[k+1][j]

else

right = solve (s, k+1, j)

tab[k+1][j] = right

int temp = 1 + left + right

}

M-2

int solve (strings, int i, int j, vector<vector<int>>> &dp)

if (tab[i][j] == -1) & return tab[i][j];

IF $(i > j)$ & returns $tab[i][j] = 0$ ✓

```
int min: INT-MAX;
```

For (int k=i; k<=j; k++) {

int tempAns = INT_MAX;

$$IF (\text{palindrome}(s, i, k)) \{$$

Temps : $t \in \text{sol}(\mathcal{L}(s, k+1, j, (a_b)))$

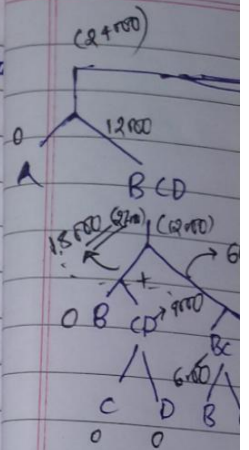
2

IF (min) Tempans

3 min: 1000s

6

```
return tab[i][j] + min;
```



~~$1 = 40 \times 20$~~

$$B = 20 \times 30$$

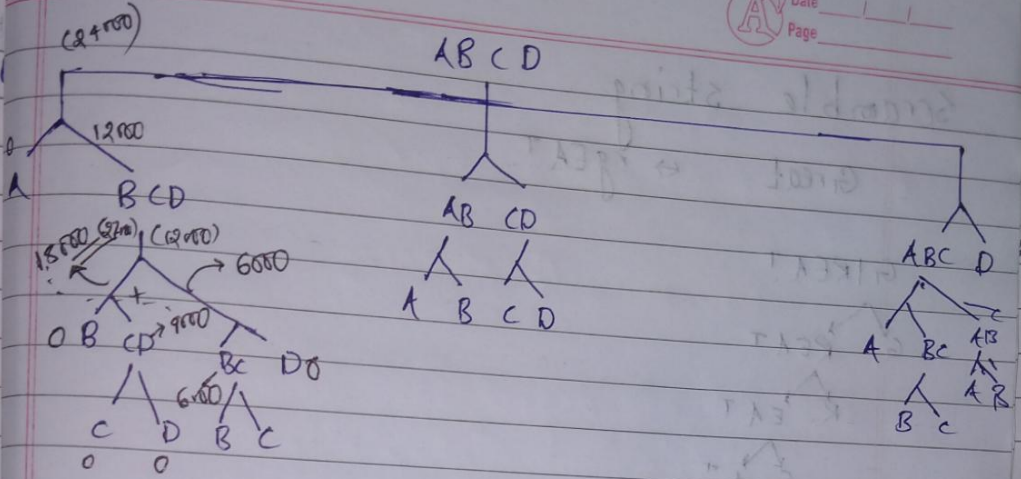
$$c = 30 \times 10$$

$$D = 10 \times 30$$



Date: / /
Page: /

vector<int>> a;
tab[0][0];



(s, k+1, i, tab, A = 40x20

B = 20x30

C = 30x10

D = 10x30

B/C/D
20x30 30x30
20x30x30

arr[] = [40 20 30 10 30]

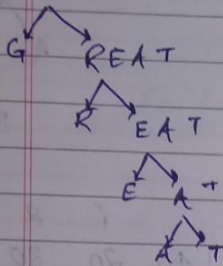
```
int MCM ( int i, int j, int arr[] )
{
    if ( i == j )
        return 0;
    int min = INT_MAX;
    for ( int k = i; k < j; k++ )
    {
        int count = MCM ( i, k, arr ) +
                    MCM ( k+1, j, arr ) +
                    arr[i-1] * arr[k] * arr[j];
        if ( count < min )
            min = count;
    }
    return min;
}

int main()
{
    int arr[] = [40, 20, 30, 10, 30];
    int n = 4;
    count < MCM ( 1, 4, arr )
}
```

Scramble string

Great $\leftrightarrow$ gEAT

GIREAT



bool solve (string s1, string s2)

{

if (s1.size() == 1)

return s1 == s2;

if (s1 == s2) return true;

string key = s1 + s2;

if (mem.find(key) != mem.end())

{ return mem[key]; }

int n = s1.size();

for (int i = 1; i < n; i++)

{

if (solve(s1.substr(0, i), s2.substr(0, i)) and
solve(s1.substr(i), s2.substr(i)))

or solve(s1.substr(0, i), s2.substr(n-i))

and solve(s1.substr(i), s2.substr(0, n-i)))

return mem[key] = true;

}

return mem[key] = false;

}

s1 $\rightarrow$ A B C D E F
 $\leftarrow x \rightarrow \leftarrow y \rightarrow$
s2 $\rightarrow$ B C D A E F
 $\leftarrow x' \rightarrow \leftarrow y' \rightarrow$

Date _____
Page _____

Date _____
Page _____

CDEF
x' - y' →
DAEF
← y' →

nd

(0, 0, 1))



Date

Page

ate
ge



Date
Page



Date _____
Page _____

1. 100 - 10 = 90
2. 100 - 20 = 80
3. 100 - 30 = 70
4. 100 - 40 = 60
5. 100 - 50 = 50
6. 100 - 60 = 40
7. 100 - 70 = 30
8. 100 - 80 = 20
9. 100 - 90 = 10
10. 100 - 100 = 0

if $\frac{1}{2} < x < 1$ then $f(x) = 2x - 1$
if $x = 1$ then $f(x) = 1$

if $x = 0$ then $f(x) = 0$
if $x < 0$ then $f(x) = 0$

if $x = 1$ then $f(x) = 1$
if $x > 1$ then $f(x) = 0$

if $x = 0$ then $f(x) = 0$
if $x < 0$ then $f(x) = 0$

if $x = 1$ then $f(x) = 1$
if $x > 1$ then $f(x) = 0$

if $x = 0$ then $f(x) = 0$
if $x < 0$ then $f(x) = 0$

if $x = 1$ then $f(x) = 1$
if $x > 1$ then $f(x) = 0$

if $x = 0$ then $f(x) = 0$
if $x < 0$ then $f(x) = 0$

if $x = 1$ then $f(x) = 1$
if $x > 1$ then $f(x) = 0$

if $x = 0$ then $f(x) = 0$
if $x < 0$ then $f(x) = 0$

if $x = 1$ then $f(x) = 1$
if $x > 1$ then $f(x) = 0$

if $x = 0$ then $f(x) = 0$
if $x < 0$ then $f(x) = 0$

if $x = 1$ then $f(x) = 1$
if $x > 1$ then $f(x) = 0$

Recursion $\Rightarrow$ using pick & not pick $\rightarrow O(2^N)$
 $SC \rightarrow O(N)$

$\Downarrow$ Memoize

```
int f (int ind, int pred_ind, int arr[], int n,
      vector<vector<int>> & dp) {
    if (ind == 0) return 0;
    if (dp[ind][pred_ind+1] != -1) return dp[ind][pred_ind+1];
```

TC $\rightarrow O(N^2)$
 SC $\rightarrow O(N^2)$

```
    int len = 0 + f(ind+1, pred_ind, arr, n, dp);
    if (pred_ind == -1 || arr[ind] > arr[pred_ind])
        len = max(len, 1 + f(ind+1, ind, arr, n, dp));
    return dp[ind][pred_ind+1] = len;
```

```
    return dp[ind][pred_ind+1] = len;
```

}

```
int longestIS (int arr[], int n) {
    vector<vector<int>> dp(n, vector<int>(n, -1));
```

more than 10⁵

10⁵ $\times$ 10⁵ = 10¹⁰

```
return f(0, -1, arr, n, dp);
```

}

it will not pass all testcases

}

Q1

Longest

[3, 10, ...]

int n

seq

for (int

{

return

for (

{ return



Date _____
Page _____

Q.1

Longest increasing subsequence

Pick $\rightarrow O(2^N)$

SC $\rightarrow O(N)$

[3, 10, 2, 1, 20] $\rightarrow$ 3, 10, 20

[50, 3, 10, 7, 40, 80] $\rightarrow$ 3, 10, 40, 80

Brute Force $\rightarrow$ check all subsequence $\rightarrow$ Power Set
 $\downarrow$
 $O(2^N)$

int n = nums.size();

vector<int> seq; seq.push_back(0, 1);

seq.push_back

for (int i = 1; i < n; i++)

$O(n^2), O(n)$

{

for (int j = 0; j < i; j++)

//

{

if (nums[i] > nums[j] && dis[i] < dis[j] + 1)

dis[i] = dis[j] + 1;

}

}

if (max < dis[i]) max = dis[i];

}

return max;

//

int n = nums.size();

vector<int> seq;

seq.push_back(nums[0]);

for (int i = 1; i < n; i++)

if (nums[i] > seq.back()) seq.push_back(nums[i]);

else

int ind = lower_bound(seq.begin(), seq.end(), nums[i]) - seq.begin();

seq[ind] = nums[i];

}

return seq.size();



Date _____
Page _____

[], ind, arr, n, dp

TC $\rightarrow O(N^2)$

SC $\rightarrow O(N)$

dp[ind]
[pred-ind+1]

arr, n, dp

arr[pred-ind]

ind, arr, n, dp

n;

move
than 10

10

= 10

<int> (arr, n)

}

if arr

ot pass all testcases

Box stacking problem

Q. Max^m sum increasing subsequence

[1, 100, 3, 2, 6]

[1, 3, 6] → 10

[1, 2, 6] → 9

[1, 100] → 101 → Ans

similar as previous

if (arr[j] > arr[i] && max[j] < max[i] + arr[j])
{
 max[j] = max[i] + arr[j]
}

The maximum value of the function
 is $\frac{1}{2}$ at $x = \frac{\pi}{2}$
 and the minimum value is $-\frac{1}{2}$ at $x = \frac{3\pi}{2}$
 The function is periodic with period 2π
 The range of the function is $[-\frac{1}{2}, \frac{1}{2}]$
 The graph of the function is a sine wave
 with amplitude $\frac{1}{2}$ and period 2π

[Misc]
 + [Misc]

The function $f(x) = \sin(x)$ is periodic with period 2π
 The range of the function is $[-1, 1]$
 The maximum value of the function is 1 at $x = \frac{\pi}{2}$
 and the minimum value is -1 at $x = \frac{3\pi}{2}$
 The function is increasing on $(-\frac{\pi}{2}, \frac{\pi}{2})$
 and decreasing on $(\frac{\pi}{2}, \frac{3\pi}{2})$
 The function is concave up on $(0, \pi)$
 and concave down on $(\pi, 2\pi)$
 The graph of the function is a sine wave
 with amplitude 1 and period 2π

RQ. Maximum sum of non-adjacent elements.

Return the maximum sum of the subsequence with the constraint that no two elements are adjacent in the given arraylist.

1 2 4

O/P: - 5

2 1 4 9

O/P: - 11

→

int traverse (int n, vector<int> nums)

{
if (n == 0) & return nums[0];

if (n < 0) & return 0;

if (tab[n] != -1) & return tab[n];

return tab[n] = max (nums[n] + traverse (n-2, nums, tab),
traverse (n-1, nums, tab));

}

RQ. House robber II → similar Q.

Note:- arrays are circularly connected means index 0 & last are connected.



Date _____
Page _____

adjacent

of the
constraint
are adjacent

or <int> nums)

tab[n];

traverse

(n-2, nums, tab,
-1, nums, tab))

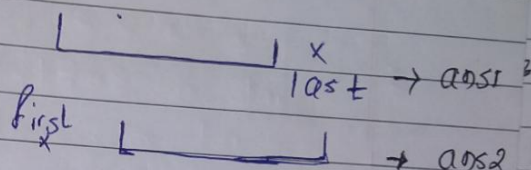
similar to

connect
are



Date _____
Page _____

Ans + First & last X



max(ans1, ans2)

long traverse (int n, num, tab, int start)

{

if (n < start) { return 0; }

if (tab[n] != -1) { return tab[n]; }

return tab[n] = max (nums[n] + traverse(n-2,
nums, tab, start),

traverse(n-1, nums, tab, start))

}

int main()

{ vector<int> tab1(n, -1), tab2(n, -1);

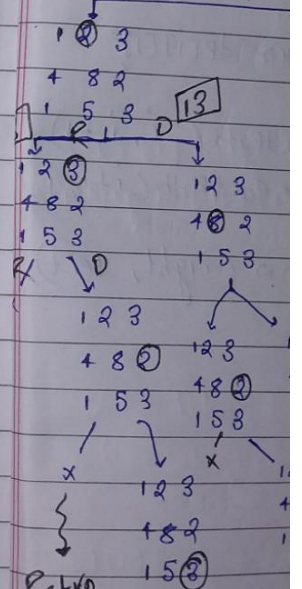
return max (traverse(n-1, nums, tab1, 1),
traverse(n-2, nums, tab2, 0))

}

1	2	3
4	8	2
1	5	8

→

☒ Right



Return 15%
Subvalue
which is
not possible

Minimum Cost Path

4 8 2

1 5 8

→ Constrain or ~~make~~
right or ~~down~~

17

① 2 3

4 8 2

1 5 3

→ 40 units

D

→

right

1 2 3

4 8 2

2

82

53

↓

12

15

1

x

ber/9

total
1.15

ent poss

 (i, j)

(ii) $\rightarrow$ Bottom

$$(i, j+1) \rightarrow \text{right}$$

```

int minCostPath(int i, int j, vector<vector<int>>
               &mat,
               vector<vector<int>> &dp)
{
    int m = mat.size();
    int n = mat[0].size();
    if (i == (m-1) && j == (n-1)) return mat[i][j];
    if (i >= m || j >= n) return INT_MAX;
    if (dp[i][j] != -1) return dp[i][j];
    int right = mat[i][j] + minCostPath(i, j+1);
    int left = mat[i][j] + minCostPath(i+1, j);
    return dp[i][j] = min(right, left);
}

```

Q. Grid Unique Path I

Q. Grid Unique Path II

Base Qⁿ

QI

Q. Ninja's

	1	2
days	3	1
Point	3	3

int

{

if (

for (

return

{

Base Q

Q1

Follow-up Qs

1. Grid unique Path
2. Grid unique Path
3. minimum cost path
4. Triangle
5. Max/min falling Path sum

Q. Ninja's Training

	days	activities	constraint
days	1	2	5
Point	3	1	1
	3	3	3

you can't do same activity on two consecutive days.

int f(int day, int last, Points, dp)

{

if (day == 0) {

int maxi = 0;

for (int task = 0; task < 3; task++) {

if (task != last) {

{

maxi = max(maxi, Point[day][task]);

}

return maxi;

}

if (dp[day][last] != -1) return dp[day][last];

int maxi = 0;

for (int task = 0; task < 3; task++) {

if (task != last) {

int point = Points[day][task];

+ f(day-1, task, Points, dp);

maxi = max(maxi, point);

}

dp[day][last] = maxi;

return

vector<vector<int>> dp(n, vector<int>(3, -1));

hard → 3D Memoization

Q. Cherry-Pick-up

↓			↓
2	3	1	2
3	4	2	2
5	6	3	5

long long int f (int i, int j1, int j2, r, c, grid, tab)

if (j1 < 0 || j2 < 0 || j1 > c || j2 > c)
return INT_MIN

if (i == r-1)

if (j1 == j2) return grid[i][j1]
else return grid[i][j1] + grid[i][j2]

int maxi = -1e8;

for (int dj1 = -1; dj1 <= +1; dj1++)

for (int dj2 = -1; dj2 <= +1; dj2++)

int value = 0;

if (j1 == j2) value = grid[i][j1]

else value = grid[i][j1] + grid[i][j2]

value += f(i+1, j1+dj1, j2+dj2, r, c)

maxi = max(maxi, value)

return maxi

vector<vector<vector<int>>> dp(r, vector<vector<int>>(c, vector<int>(c-1)))

grid, tab)

11 12 20

[i][j]

grid[i][j]

++X

++Y

d[i][j]

+ grid[i][j]

12 + d[12, 10]

100 < m1 >

- 1) 12



Date

Page

① Best time

② Best time

* Learn

⇒ you can
at a

Prices[] = [

long long f (

if

if (dp[i]

long

if (bu

Profit

6

2150

5 Pro

6

return

2

return

Buy & selling ^{can} only be done
+ once

① Best time to buy and sell stock

arr[] $\rightarrow$ [7, 1, 5, 3, 6, 4] O/P = 5

② Best time to buy and sell stock II

* Learn space optimization $\rightarrow$ They expect it

$\Rightarrow$ you can buy & sell multiple times but
at a time you can hold only one

Prices[] = [7, 1, 5, 3, 6, 4] O/P = -7

long long f (int ind, int buy, long * values, int
vector & vector < long > & dp)

if (ind == n) $\&$ return 0;

if (dp[ind][buy] != -1) $\&$ return dp[ind][buy];

long Profit = 0; Memo $\rightarrow$ $O(N \times 2)$

if (buy)

SC $\rightarrow O(N \times 2) + O(N)$

Simple recursion $\rightarrow O(2^N)$ auxiliary

Profit = max(-values[ind] + f(ind+1, 0, values, n, dp),

0 + f(ind+1, 1, values, n, dp))

&

Profit = max(values[ind] + f(ind+1, 1, values, n, dp),

0 + f(ind+1, 0, values, n, dp))

&

return dp[ind][buy] = Profit;

vector < vector < long > > dp (n, vector < long > (-1))

return f(0, values, n, dp);

Table

vector <vector<long>> dp(n+1, vector<long>
(2, 0));

dp[0][0] = dp[0][1] = 0;

↑ reverse of bottom up

for (int ind = n-1; ind >= 0; ind--) {

for (int buy = 0; buy <= 1; buy++) {

long profit = 0;

if (buy) {

profit = max(-prices[ind] + dp[ind+1][0],
0 + dp[ind+1][1]);

else
{

profit = max(prices[ind] + dp[ind+1][0],
0 + dp[ind+1][1]);

}

dp[ind][buy] = profit;

}

return dp[0][1];

③ Best time to buy & sell stock III

→ Use 3D DP → Memoization TE

Use table approach,

to, <long>

(2, 0)

) 5

64x + 12

dp[ind+1][0]

j[1]

+ dp[ind+1][0]

ind+1][0]

stock III

ation TLE



Date _____
Page _____